

智能软件开发 方向基础



第五章 决策树 decision tree

第3部分 决策树剪枝 张朝晖

2023~2024学年第二学期



序号	内容
1	概述
2	机器学习的基本概念
3	模型的选择与性能评价
4	数据的获取、探索与准备
5	近邻模型-----分类、回归
6	决策树模型-----分类、回归
7	集成学习-----分类、回归
8	(朴素)贝叶斯模型-----分类
9	聚类
10	特征降维及低维可视化(PCA, t-SNE)
11	总复习

主要内容

决策树

基于树形结构的决策模型——决策树

包括：决策树构建方法；决策树的剪枝；决策树的使用

1 非度量特征(nonmetric features)

2 初步认识决策树

3.决策树的构建

3.1 面向分类问题的决策树特征选择

3.2 分类树的构建

3.3 回归树的构建

4.过学习与决策树的剪枝



(1)模型的过拟合(overfitting)和欠拟合(underfitting)

➤ 分类模型的误差：

训练误差，是训练样本集内错分样本占比

泛化误差，是模型关于未知样本分类的期望误差

训练误差越低，模型的**学习能力**越好；

泛化误差越低，模型的**推广能力**越强

➤ 好的分类模型应具有**低训练误差**和**低泛化误差**。

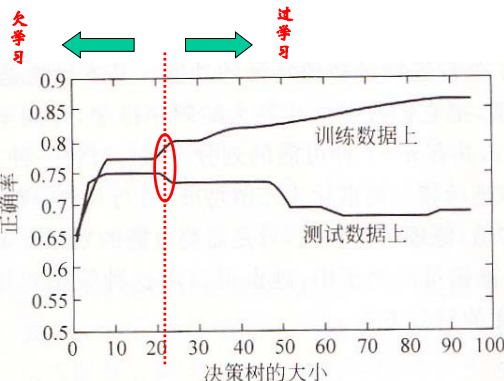
➤ 具有**较低训练误差**的模型，其**泛化误差**可能**高于**具有**较高训练误差**的模型，这种情况称为**模型过拟合(过学习)**



➤ **决策树规模很小时**，训练和检验误差都很大，这种情况为**模型的欠拟合(欠学习)**，原因是**模型尚未学习到数据的真实结构**。

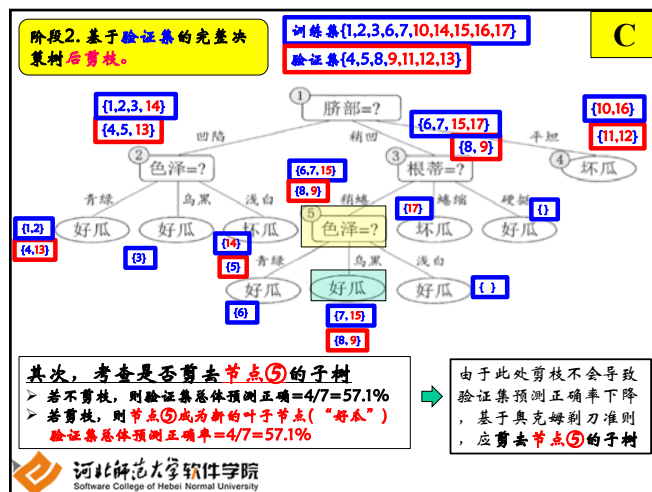
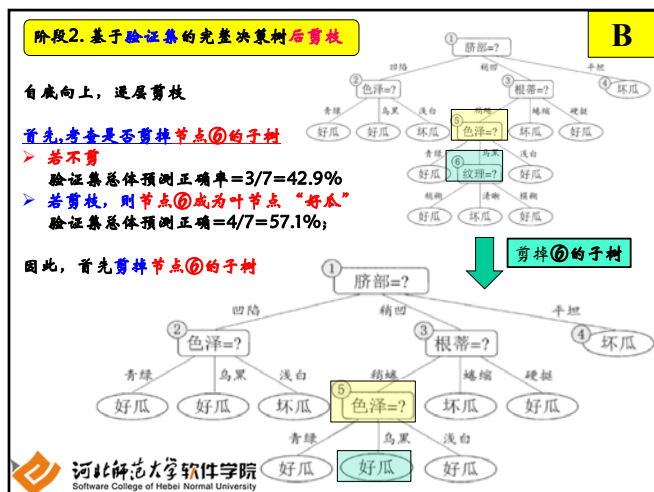
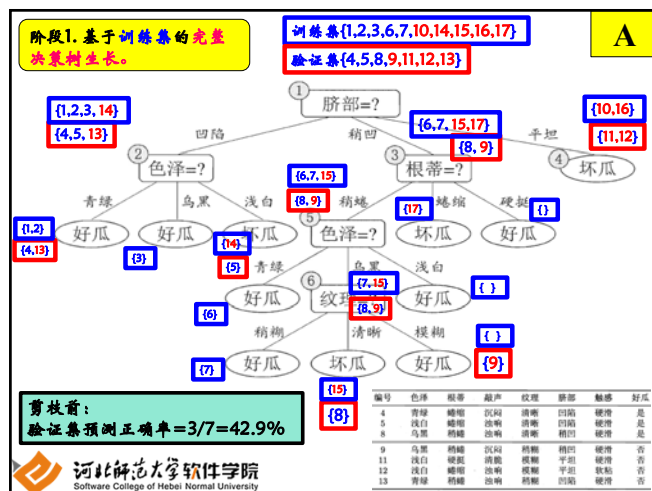
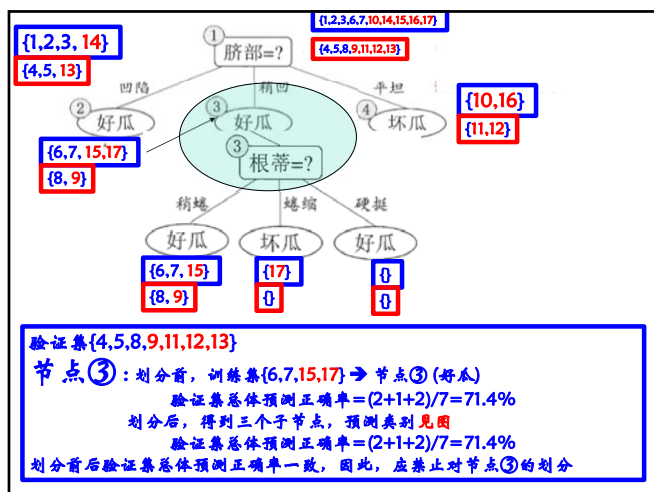
➤ **随着决策树节点数的增加**，模型的训练误差和检验误差都会随之下降。当树的规模变得太大时，即使训练误差还在继续降低，但是检验误差开始增大，导致**模型过拟合(过学习)**，其原因在于**过分关注采样偶然性或噪声等因素影响**。

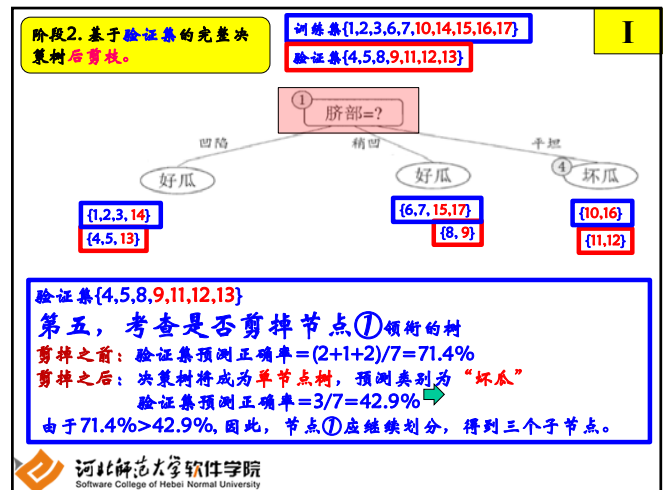
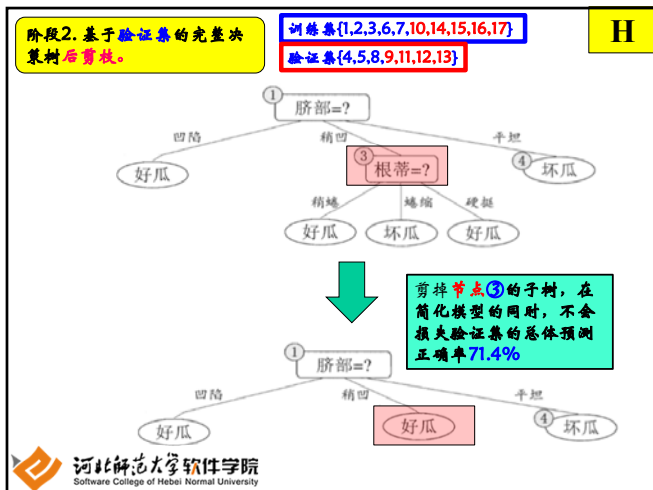
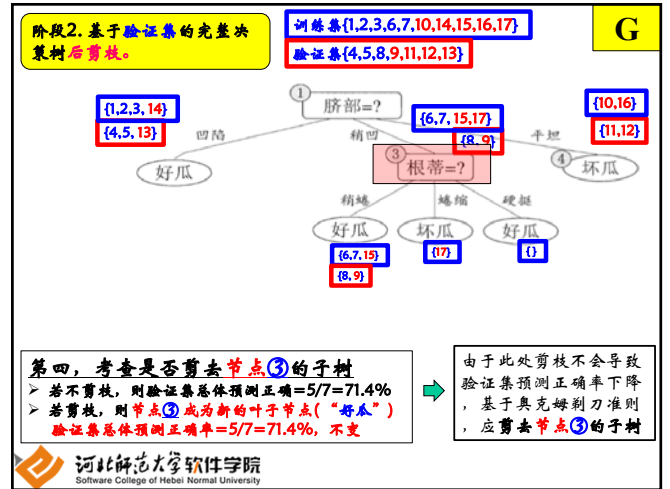
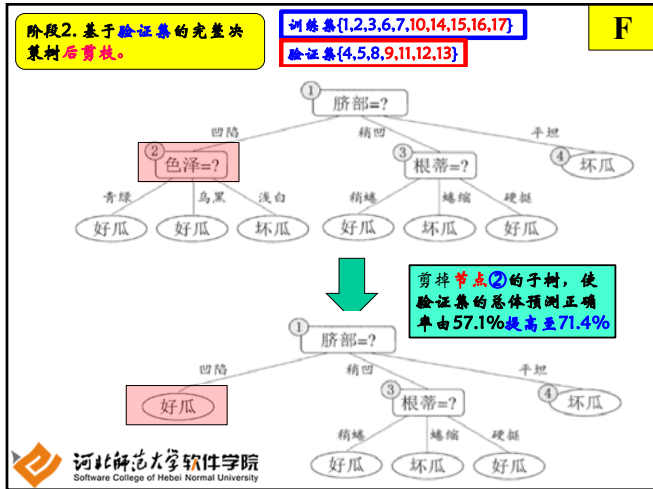
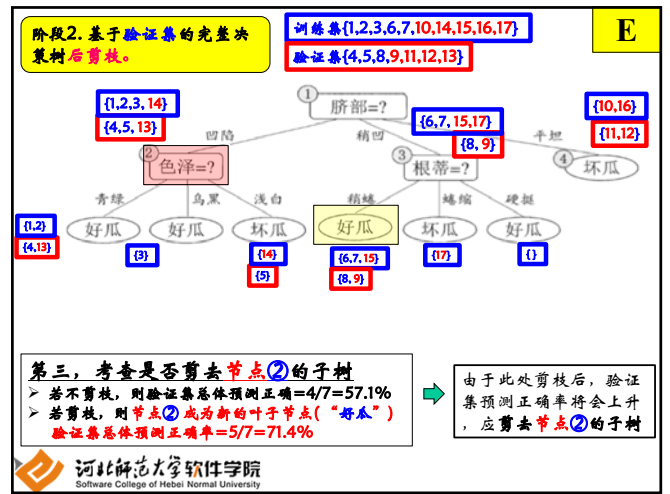
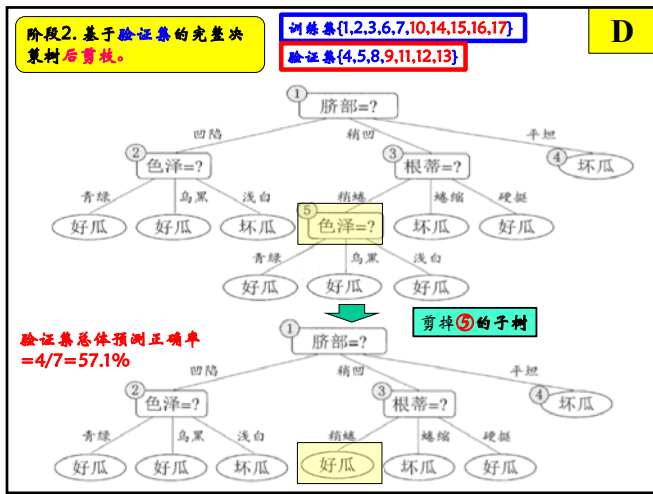
➤ 若训练数据**缺乏具有代表性的样本**，并且**样本规模较小**，模型也会产生**过拟合**。



ID3决策树的过拟合现象

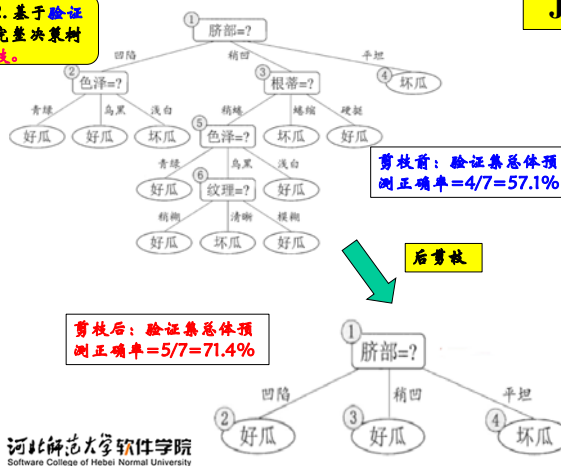






阶段2: 基于验证集的完整决策树后剪枝。

J



策略2: 后剪枝(post-pruning)

剪枝规则

例: 一种后剪枝算法--最小代价与复杂性的折中:
平衡“错误率的增加”与“模型复杂程度的降低”

决策树的后剪枝 (1)

设决策树 T 的叶节点数目为 $|T|$, 叶节点序号 $t = 1, \dots, |T|$
训练样本集到达叶节点 t 的样本数为 N_t ,
其中第 k 类的样本数为 $N_{tk}, k = 1, \dots, K$.

叶节点 t 的不纯度 $H_t(T)$ $H_t(T) = - \sum_{k=1}^K \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t}$

决策树 T 关于训练样本集的拟合误差
(训练集划分的综合不纯度):

$$C(T) = \sum_{t=1}^{|T|} N_t H_t(T) = - \sum_{t=1}^{|T|} N_t \left(\sum_{k=1}^K \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t} \right)$$

$$= - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log \frac{N_{tk}}{N_t}$$

决策树的后剪枝 (2)

决策树 T 关于训练样本的拟合误差:

$$C(T) = \sum_{t=1}^{|T|} N_t H_t(T) = - \sum_{t=1}^{|T|} N_t \sum_{k=1}^K \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t} = - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log \frac{N_{tk}}{N_t}$$

模型的损失函数=模型关于训练样本的拟合误差 + 模型的复杂度

模型的损失函数: $C_\alpha(T) = C(T) + \alpha |T|$

决策树的后剪枝, 就是在给定 $\alpha \geq 0$ 的前提下,
选择具有最小 $C_\alpha(T)$ 的子树。

决策树 T 的后剪枝算法

输入: 生成算法产生的整棵树 T , 控制参数 α

输出: 对树 T 修剪, 得到的子树 T_α

步骤:

STEP1. 基于训练集计算每个节点 (不只是叶节点) 的经验熵。

STEP2. 递归地从树的叶节点向上回溯。

设一组叶节点回溯到其父节点之前、之后的整体树分别为 T_{Before} 和 T_{After} ; 对应的损失函数值分别为

$$C_\alpha(T_{Before}) = C(T_{Before}) + \alpha |T_{Before}|$$

$$C_\alpha(T_{After}) = C(T_{After}) + \alpha |T_{After}|$$

若 $C_\alpha(T_{After}) \leq C_\alpha(T_{Before})$, 则剪枝, 将叶节点的父节点作为新的叶节点

STEP3. 返回STEP2, 直到不能继续为止, 得到损失函数最小的子树 T_α

(3) 关于“剪枝”的讨论:

➤ 预剪枝可能过早终止决策树的生长
存在欠拟合风险。

➤ 后剪枝倾向于产生更好的结果

根据完全生长的决策树作出剪枝决策, 需要更多时间开销。

欠拟合的风险小, 泛化性能优于预剪枝决策树。

➤ “预剪枝”与“后剪枝”结合